

OPTAM-MPC

Hands-on Application Tutorial

Part 1: Orientation and the Linear Surge Vessel

A guided bridge from the frozen v0.6.0 manual to your first independently configured MPC application

Applies to	OPTAM-MPC Linear Starter v0.6.0
Audience	APC, process-control and automation engineers
Issue	Draft 1 — 6 September 2026
Scope	Modules 1 and 2 only

This tutorial does not modify the frozen software foundation.

How to use this tutorial

This is a guided engineering exercise, not a second user manual. Keep the frozen user manual beside you as the authoritative reference. The tutorial explains the reasoning, asks you to predict behaviour before running it, and then uses numerical evidence to test your understanding.

FROZEN-FOUNDATION RULE Do not edit the OPTAM_MPC_Linear_Starter_v0.6.0 ZIP file. Extract a fresh working copy and create your application in a separate folder.

What you will achieve

- Recognise the boundary between the engineer-owned application and the trusted controller engine.
- Translate a physical surge-vessel material balance into an OPTAM-MPC linear integrator path.
- Configure one measured CV, one internal rate CV and one MV without writing Python.
- Represent a wild inlet as a hidden simulation disturbance—not as a measured controller DV.
- Validate, simulate and judge the application using pre-agreed numerical acceptance checks.
- Run the same engineered application through the local OPC UA twin using generic commands.

What this part deliberately does not teach

- Model identification from plant data.
- Python programming or modification of the OPTAM-MPC engine.
- Nonlinear equations, multiple MVs or measured-disturbance feedforward.
- Plant commissioning, safety-system design or unattended industrial deployment.

WORKING METHOD For every important choice: predict first, configure second, validate third, run fourth, and record what the evidence actually shows.

Module 1 — Whole-system orientation

1.1 Start with the destination

At the end of an OPTAM-MPC application-development exercise, you do not merely have a controller that ran once. You have an engineering record: a defined control problem, a reviewed model, explicit tuning, an independent test plant, disturbance scenarios, acceptance checks, numerical reports and a reproducible runtime configuration.

The application workflow at a glance

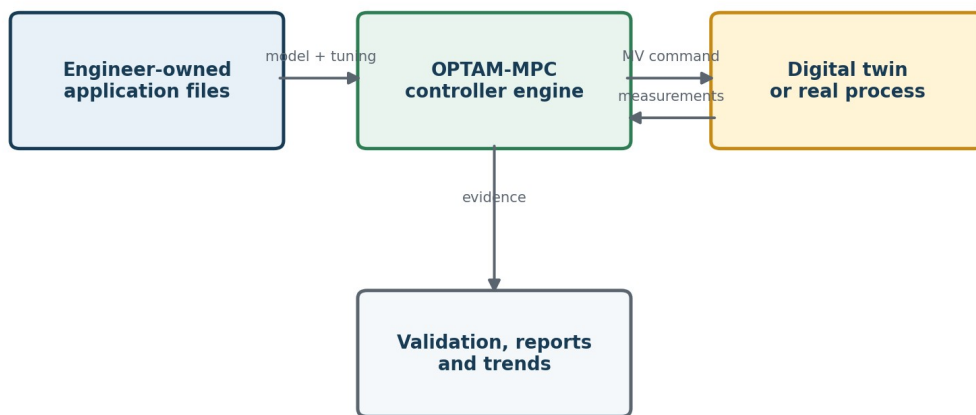


Figure 1. Engineer-owned information surrounds—but does not alter—the generic controller engine.

1.2 Four application files

File	Engineering purpose
application.toml	Defines CVs, MVs, measured DVs, the controller model, horizons, limits and tuning.
simulation.toml	Defines an independent test plant, hidden disturbances, scenarios, plots and numerical acceptance checks.
opc.toml	Defines the supervised local OPC UA test connection and timing.
model.py	Used only by nonlinear applications. It is not required for this linear tutorial.

1.3 Who owns what?

Engineer owns	OPTAM-MPC supplies
The control objective and variable selection	Generic optimisation engine
A working dynamic model and its validity basis	Configuration validation
Units, scaling, limits and tuning	MV limit enforcement and CV objectives
Independent test scenarios and acceptance criteria	Simulation, plotting and reports
Site commissioning and fallback philosophy	Local OPC UA development twin

IMPORTANT BOUNDARY OPTAM-MPC verifies that your model is implemented consistently. It does not identify the model and cannot prove that the model represents the real plant.

1.4 The normal development path

1. Write the design basis before editing configuration files.
2. Choose the simplest adequate model form: linear or nonlinear.
3. Copy the generic templates into a new application folder.
4. Configure the application model and independently check its implementation.
5. Configure a separate digital twin and pre-agreed acceptance criteria.
6. Validate before every run.
7. Run offline, inspect the trends and read the numerical report.
8. Challenge the controller with mismatch, disturbances and constraints.
9. Run the accepted configuration through the local OPC UA twin.
10. Preserve the exact application files and reports used for acceptance.

1.5 PASS does not mean plant ready

A PASS means only that the configured scenario satisfied the checks you defined. If the model is wrong, the scenario is too gentle, or the acceptance criteria are weak, a PASS can still be misleading. The quality of the evidence depends on the quality of the engineering challenge.

PAUSE AND PREDICT What could pass in a simulation while still failing on a real plant? Write at least three possibilities before continuing.

- A hidden disturbance could be larger or faster than the test event.
- An actuator could have stiction, rate limits or unavailable range not represented in the twin.

- The process model could have the wrong gain, dead time or operating-point dependence.
- Measurements could be noisy, biased, stale or temporarily invalid.

Module 1 engineering record

Before moving on, record short answers in `engineering_record.md`:

- What problem will the MPC solve?
- Which information is supplied by the engineer?
- Which functions are supplied by OPTAM-MPC?
- What does a simulation PASS establish—and what does it not establish?

INDEPENDENT LEARNER NOTE The unused space on this page is intentional. Pause here and write your answers before starting the vessel application.

Module 2 — Build the linear surge-vessel application

2.1 The physical problem

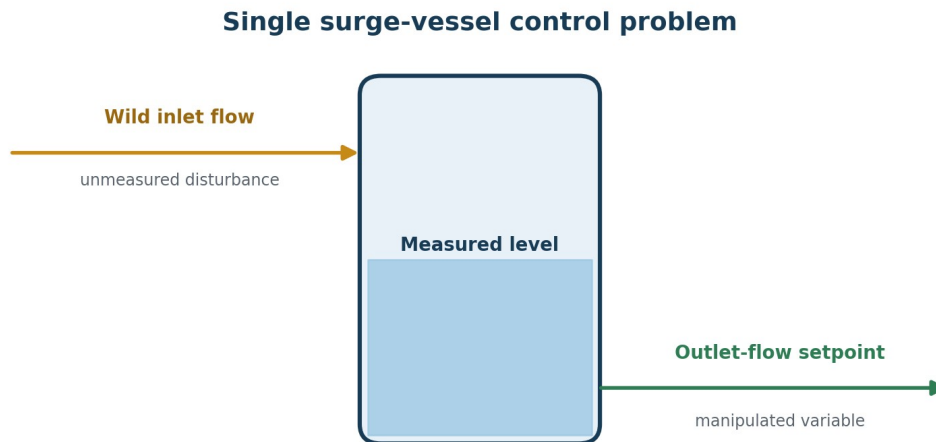


Figure 2. The inlet disturbance exists in the test plant but is intentionally hidden from the controller.

The vessel is a buffer. Its job is not to hold level perfectly at 50%. Its job is to absorb a varying inlet while providing a smoother outlet to the downstream process. Level must remain inside the usable inventory range, outlet movement should be small, and level should return slowly toward the midpoint so that future surge capacity is restored.

Control element	Tutorial definition
Measured CV	Vessel level (%)
Internal CV	Filtered level rate (%/min), target zero
MV	Outlet-flow setpoint (m^3/h)
Hidden disturbance	Wild inlet flow (m^3/h), unavailable to the controller
Protected range	20% to 80% level
Inventory objective	Slow return toward 50%
Smoothing objective	Penalise outlet-flow movement

2.2 Predict the desired response

PAUSE AND PREDICT The inlet rises from 60 to 75 m³/h, but the controller cannot measure it. Should the outlet immediately jump to 75 m³/h? Explain what a good surge response should look like.

A good averaging response initially allows level to rise, because the vessel is doing useful buffering work. The controller detects the resulting level movement and gradually increases outlet flow. Once inlet returns to 60 m³/h, outlet also returns gradually while level is restored toward 50%. Tight level control would simply transmit the inlet disturbance downstream.

2.3 Derive the first-principles model

For a constant-cross-section vessel:

$$dL/dt = (F_{in} - F_{out}) / (60 \times V_1\%)$$

Here level L is in percent, flows are in m³/h, time is in minutes, and V₁% is the volume represented by one percentage point. For this exercise V₁% = 4.0 m³/%. A +1 m³/h change in outlet flow therefore produces:

$$K_i = -1 / (60 \times 4) = -0.0041666667 \text{ (\%/min)/(m}^3\text{/h)}$$

INDEPENDENT CHECK A +5 m³/h outlet step must give -0.0208333333 %/min. A -5 m³/h step must give +0.0208333333 %/min.

The controller model includes only the outlet-to-level path. The wild inlet is not included as a measured DV because the application premise says it is unavailable. The independent test plant includes both inlet and outlet paths.

2.4 Create a clean workspace

Extract the frozen Linear Starter into a new writable folder, then run:

```
setup_windows.bat
```

Create a separate application folder and copy the generic templates:

```
mkdir applications\my_surge_vessel
copy application_template.toml applications\my_surge_vessel\application.toml
copy simulation_template.toml applications\my_surge_vessel\simulation.toml
copy opc_template.toml applications\my_surge_vessel\opc.toml
```

NO SHORTCUTS Do not copy the completed surge-vessel example yet. The objective is to bridge from the generic templates to a functioning application using your engineering decisions.

2.5 Configure application.toml

Application and horizons

```
[application]
name = "Single surge vessel"
sample_time_seconds = 5.0
model_type = "linear_paths"

[controller]
prediction_form = "velocity"
prediction_horizon_samples = 60
control_horizon_samples = 12
maximum_solver_cpu_seconds = 4.0
```

The 60-sample prediction horizon represents 5 minutes. The 12-sample control horizon represents 1 minute. These are the values in the frozen reference application; they are not universal tuning rules.

Primary level CV

```
[[cvs]]
id = "level"
name = "Vessel level"
unit = "%"
source_type = "instrumented_process"
initial = 50.0
target = 50.0
weight = 0.20
normalization = 10.0
protected_low = 20.0
protected_high = 80.0
protected_limit_weight = 1000.0
protected_limit_normalization = 5.0
display = "operator_and_engineering"
```

Delete the second generic CV block. The relatively low level weight and 10-percentage-point normalization allow useful inventory movement. The protected-limit penalty is much stronger, but these protected limits remain soft optimisation constraints—not safety trips.

Internal filtered level-rate CV

```
[[derived_cvs]]
id = "level_rate"
name = "Filtered level rate"
unit = "%/min"
source_type = "derived_auxiliary"
calculation = "rate_of_change"
source_cv = "level"
filter_alpha = 0.30
initial = 0.0
target = 0.0
weight = 0.01
normalization = 0.05
display = "engineering"
```

This internal CV gives the controller a controlled measure of level movement. It provides proportional-like response to an unmeasured flow imbalance while filtering measurement noise. It does not turn the hidden inlet into a measured DV.

Outlet-flow MV

```
[[mvs]]
id = "outlet_flow"
name = "Outlet-flow setpoint"
unit = "m3/h"
initial = 60.0
lower = 20.0
upper = 100.0
maximum_increase_per_sample = 0.25
maximum_decrease_per_sample = 0.25
move_weight = 1.0
move_normalization = 1.0
```

Delete the second generic MV block and both measured-disturbance blocks. The outlet is intentionally prevented from moving more than 0.25 m³/h per five-second sample.

Controller model and independent checks

```
[[model_paths]]
source = "outlet_flow"
destination = "level"
type = "integrator"
integrating_gain = -0.004166666666666667
dead_time_seconds = 0.0
engineering_basis = "V_1_percent = 4.0 m3/%"

[[model_checks]]
source = "outlet_flow"
destination = "level"
step = 5.0
expected_rate_per_minute = -0.020833333333333332
tolerance = 1.0e-12

[[model_checks]]
source = "outlet_flow"
destination = "level"
step = -5.0
expected_rate_per_minute = 0.020833333333333332
tolerance = 1.0e-12
```

CHECKPOINT Your application.toml should now contain one primary CV, one derived CV, one MV, no measured DVs, one model path and two model checks.

2.6 Configure simulation.toml

The digital twin is intentionally separate from the controller model. It starts at equilibrium: inlet = outlet = 60 m³/h and level = 50%. At sample 30 the hidden inlet rises to 75 m³/h; at sample 180 it returns to 60 m³/h.

```
[simulation]
samples = 1050
maximum_solver_failures = 0

[[plant_outputs]]
id = "vessel_level"
cv = "level"
unit = "%"
initial = 50.0
nominal_rate_per_minute = 0.0

[[plant_inputs]]
id = "outlet"
role = "mv"
controller_id = "outlet_flow"
unit = "m3/h"
initial = 60.0

[[plant_inputs]]
id = "inlet"
role = "hidden"
unit = "m3/h"
initial = 60.0

[[plant_paths]]
source = "outlet"
destination = "vessel_level"
type = "integrator"
integrating_gain = -0.004166666666666667
dead_time_seconds = 0.0

[[plant_paths]]
source = "inlet"
destination = "vessel_level"
type = "integrator"
integrating_gain = 0.004166666666666667
dead_time_seconds = 0.0

[[events]]
sample = 30
input = "inlet"
value = 75.0

[[events]]
sample = 180
input = "inlet"
value = 60.0
```

Define acceptance before running

Check	Acceptance limit
Solver failures	0
Minimum level	At least 20%
Maximum level	At most 80%
Largest outlet move	No more than 0.2500001 m ³ /h/sample
Final level error	No more than 0.50 percentage points from 50%
Final outlet error	No more than 0.25 m ³ /h from 60 m ³ /h

PAUSE AND PREDICT Before running: sketch the expected level and outlet trends. Which moves first? Why does level rise? Why should outlet eventually return close to 60 m³/h?

2.7 Validate before every run

```
validate_application.bat applications\my_surge_vessel\application.toml
applications\my_surge_vessel\simulation.toml
```

Do not bypass validation. A validation failure is useful evidence: read the exact message, correct one issue, and validate again. Common exercise mistakes are leaving a second generic CV or MV block, leaving measured DVs that do not exist, using inconsistent IDs, or omitting one of the two plant paths.

HONEST FAILURE CHALLENGE Temporarily change the positive outlet-step model check to +0.0208333 %/min. Validate. Explain why rejection is correct, then restore the negative sign.

2.8 Run and interpret the offline simulation

```
run_simulation.bat applications\my_surge_vessel\application.toml
applications\my_surge_vessel\simulation.toml
```

Inspect both the plots and the machine-readable report. For the frozen reference configuration, the reported evidence is:

Result	Frozen reference value
Level range	49.999479% to 50.590742%

Result	Frozen reference value
Final level	49.999479%
Largest outlet move	0.057948496 m ³ /h/sample
Final outlet	60.217877 m ³ /h
Solver failures	0
Acceptance	PASS

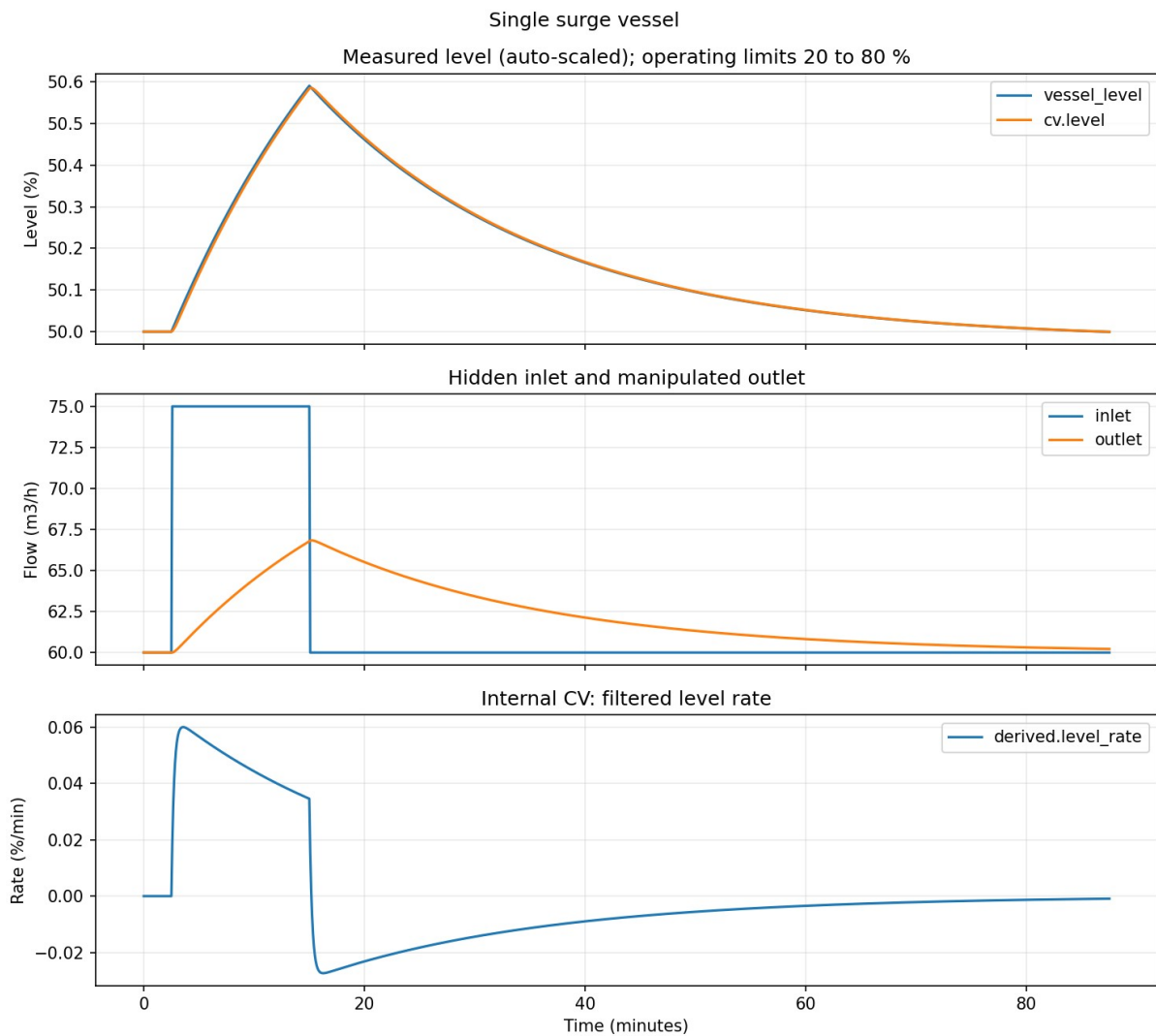


Figure 3. Frozen reference response. The outlet disturbance is smoothed while level uses a small amount of inventory capacity.

INTERPRETATION The small level excursion is not automatically good or bad. It follows from this disturbance magnitude, horizon and tuning. The key question is whether the response meets the agreed buffering objective and constraints.

2.9 Learn by changing one thing at a time

Experiment	Question to answer
Increase level weight from 0.20 to 2.0	Does level move less? What happens to outlet movement and downstream disturbance transmission?
Increase MV move weight from 1.0 to 10.0	Does the outlet become smoother? How much more inventory capacity is used?
Remove the derived level-rate CV	Does the response to the unmeasured imbalance become slower or qualitatively different?
Increase inlet event to 90 m ³ /h	Do the original horizons, MV limits and acceptance criteria remain adequate?
Change the twin vessel capacity only	How robust is the controller to a plausible model mismatch?

EXPERIMENTAL DISCIPLINE Change one item, predict the direction of the effect, run the case, record the result, and restore the accepted baseline before the next experiment.

2.10 Run the same application through OPC UA

Copy the generic `opc_template.toml` to your application folder unchanged. Then validate all three files:

```
validate_application.bat applications\my_surge_vessel\application.toml
applications\my_surge_vessel\simulation.toml applications\my_surge_vessel\opc.toml
```

Open two Command Prompt windows in the Linear Starter folder.

Window 1 — digital twin

```
run_digital_twin.bat applications\my_surge_vessel\application.toml
applications\my_surge_vessel\simulation.toml applications\my_surge_vessel\opc.toml
```

Wait for Twin READY.

Window 2 — controller

```
run_controller.bat applications\my_surge_vessel\application.toml applications\my_surge_vessel\opc.toml
```

Optional Window 3 — live trends

```
opc_tools.bat applications\my_surge_vessel\opc.toml trend applications\my_surge_vessel\simulation.toml
```

The OPC test should reproduce the same engineering response as the offline case. Equality between the two routes is evidence that transport and process separation have not changed the application logic; it is not evidence that an external plant OPC interface is commissioned.

2.11 Completion checklist

- I created the application from generic templates rather than relying on a vessel-specific launcher.
- I can derive and independently check the integrator gain.
- I understand why the wild inlet is hidden in the twin and absent from the controller DVs.
- I understand why surge-vessel level is permitted to move.
- I validated before running and read the numerical acceptance report.
- I changed one tuning item at a time and recorded predicted versus observed effects.
- I ran the same files through the local OPC UA twin.
- I can explain what the resulting PASS does and does not prove.

Where to go next

Stop here for the first independent-engineer trial. The immediate objective is not to add more subject matter. It is to discover whether a competent APC engineer can complete these two modules without help from the development team. Record every point of confusion, failed instruction and unjustified assumption.

Evidence to collect	Why it matters
Time to first successful validation	Reveals setup and configuration friction.
Questions asked	Reveals assumptions that are obvious only to the developers.
Validation failures	Shows whether messages guide correction effectively.
Incorrect engineering choices	Shows where explanation or exercises are insufficient.
Ability to interpret trends and PASS	Tests understanding rather than button-pushing.
Suggested wording and workflow changes	Provides the basis for Draft 2.

PROPOSED PART 2 After this trial: the 2×2 linear heat exchanger, measured-DV feedforward, scaling, interaction, mismatch and detuning. The nonlinear rebuild should follow only after the linear workflow is understood.

Appendix A — Files supplied with this tutorial

- 00_blank_templates: untouched generic templates copied from Linear Starter v0.6.0.
- 90_completed_reference: completed surge-vessel application, simulation and OPC configuration for comparison after the independent attempt.
- engineering_record.md: a simple record for predictions, evidence, failures and conclusions.
- SOURCE_AND_VERSION.txt: identifies the frozen package to which the tutorial applies.

Appendix B — Reference command summary

```

setup_windows.bat
validate_application.bat <application.toml> <simulation.toml> [opc.toml]
run_simulation.bat <application.toml> <simulation.toml>
run_digital_twin.bat <application.toml> <simulation.toml> <opc.toml>
run_controller.bat <application.toml> <opc.toml>
opc_tools.bat <opc.toml> trend <simulation.toml>
opc_tools.bat <opc.toml> stop

```

Appendix C — Minimum vocabulary

Term	Meaning in this tutorial
CV	A variable the controller is asked to regulate, target or keep within a zone.
MV	A variable the controller is permitted to change.
Measured DV	A measured influence used for prediction but not manipulated.
Hidden disturbance	An influence present in the test plant but unavailable to the controller.
Prediction horizon	How far ahead the controller evaluates expected behaviour.
Control horizon	How many future MV positions are independently optimised.
Normalization	An engineering error or move scale used to make weights comparable.
Velocity form	Incremental prediction anchored to successive process measurements.